Graph Algorithms & Data Structures

(cid:87)(cid:101)(cid:108)(cid:99)(cid:111)(cid:109)(cid:101) (cid:116)(cid:111) §§§§¶¶¶¶ (cid:65)(cid:10

A graph is a non-linear data structure consisting of vertices and edges.

#$%^&*!@~_+-= (cid:45)(cid:67)(cid:89) >>>>> <<<<<<

Breadth First Search (BFS) traverses a graph level by level using a queue. (cid:12)(cid:13)(cid:14)(cid:15) ···· (cid:99)(cid:98)(cid:97)

Depth First Search (DFS) traverses along each branch using recursion or a stack.

Dijkstra algorithm computes shortest paths in weighted graphs with non-negative edge weights.

Topological sorting orders vertices in a directed acyclic graph linearly.

Binary Search Trees and Balanced Trees

(cid:87)(cid:101)(cid:108)(cid:99)(cid:111)(cid:109)(cid:101) (cid:116)(cid:111) §§§§¶¶¶¶ (cid:65)(cid:10

A Binary Search Tree (BST) maintains the property that left children are smaller and right children larger.

#$%^&*!@~_+-= (cid:45)(cid:67)(cid:89) >>>>> <<<<<<

AVL trees maintain balance by ensuring the height difference between subtrees is at most one. (cid:12)(cid:13)(cid:14)(cid:15) (cid:99)(cid:98)(cid:97)

Red-Black Trees maintain balance through node color properties and tree rotations.

Tree traversals include Inorder, Preorder, and Postorder sequences.

Dynamic Programming and Optimization

(cid:87)(cid:101)(cid:108)(cid:99)(cid:111)(cid:109)(cid:101) (cid:116)(cid:111) §§§§¶¶¶¶ (cid:65)(cid:10

....

Dynamic programming solves complex problems by breaking them into overlapping subproblems.

#$%^&*!@~_+-= (cid:45)(cid:67)(cid:89) >>>>> <<<<<<

Optimal substructure means optimal solutions contain optimal solutions to subproblems. (cid:12)(cid:13)(cid:14)(cid:15) (cid:99)(cid:98)(cid:97)

Memoization stores results of expensive function calls to avoid recomputation.

Tabulation builds the solution iteratively from the bottom up.